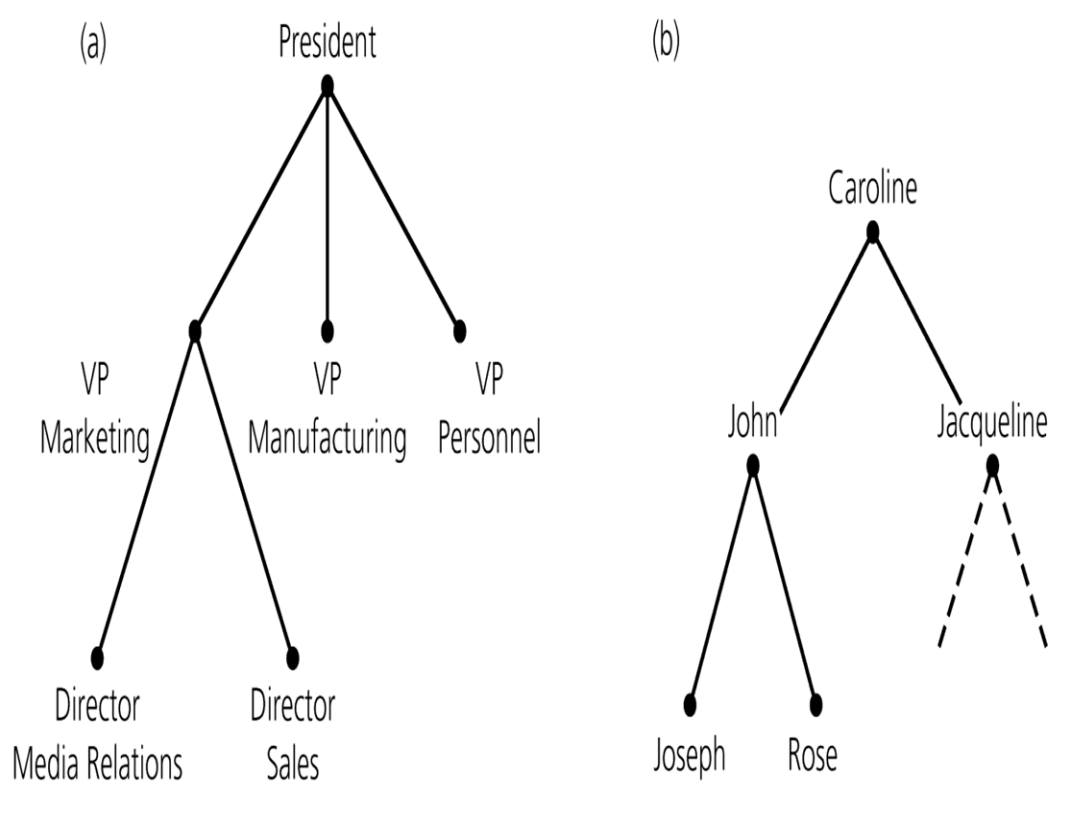
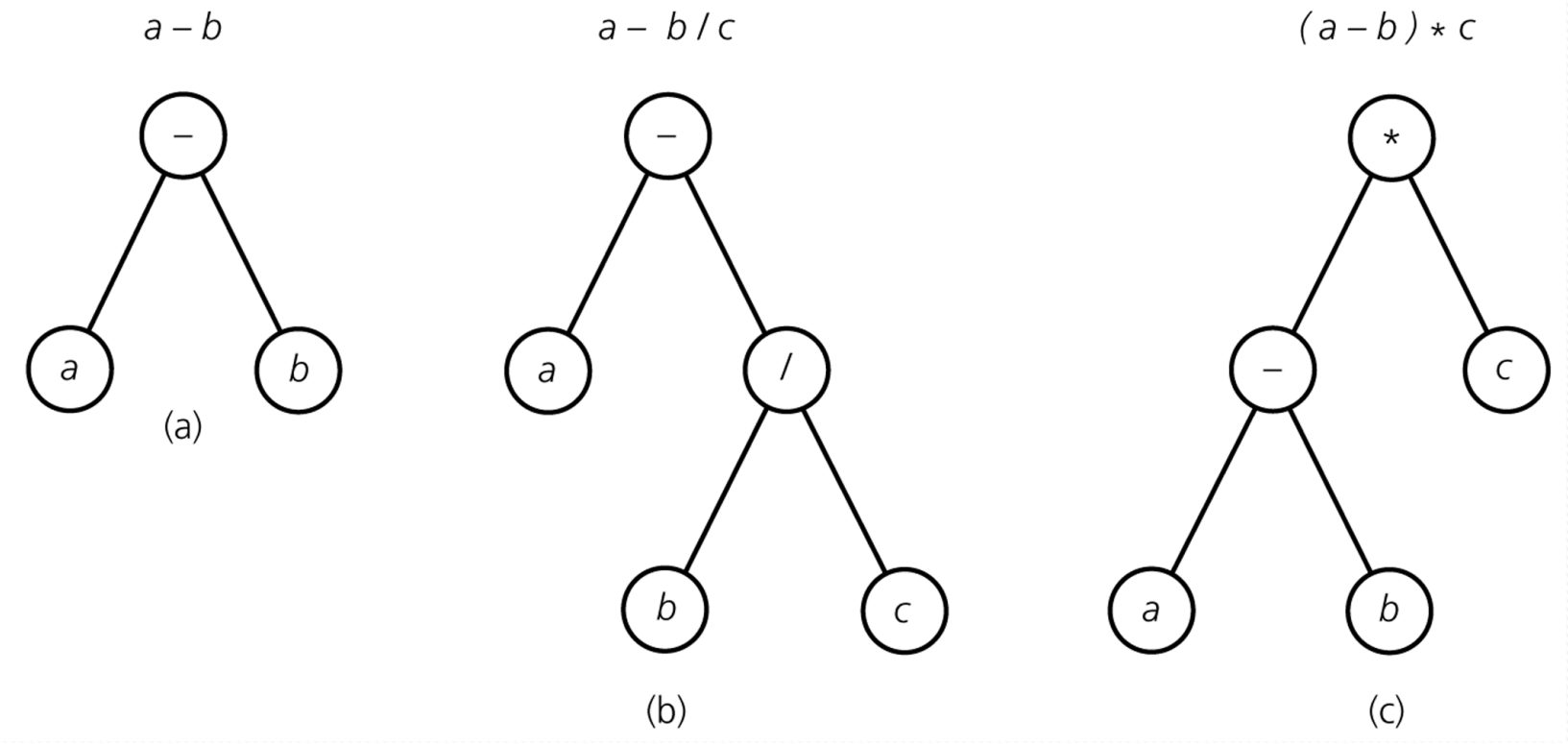
Tree

-Dr. Shital Dongre


(a)
President
VP
Marketing
VP
Manufacturing
VP
Personnel
Director
Media Relations
Director
Sales
(b)
Caroline
John
Jacqueline
Joseph
Rose


a – b
–
a
b
(a)
a – b / c
–
a
/
b
c
(b)
(a – b) * c
*
–
c
a
b
(c)

Introduction to Tree

- Fundamental data storage structures used in programming.
- Combines advantages of an ordered array and a linked list.
- Searching as fast as in ordered array.
- Insertion and deletion as fast as in linked list.

Tree characteristics

- Consists of *nodes* connected by *edges*.
- Nodes often represent entities (complex objects) such as people, car parts etc.
- Edges between the nodes represent the way the nodes are related.
- Its easy for a program to get from one node to another if there is a line connecting them.
- The only way to get from node to node is to follow a path along the edges.

Tree Terminology

- Path: Traversal from node to node along the edges results in a sequence called path.
- Root: Node at the top of the tree.
- Parent: Any node, except root has exactly one edge running upward to another node. The node above it is called parent.
- Child: Any node may have one or more lines running downward to other nodes. Nodes below are children.
- Leaf: A node that has no children.
- Subtree: Any node can be considered to be the root of a subtree, which consists of its children and its children's children and so on.

Tree Terminology

- Visiting: A node is visited when program control arrives at the node, usually for processing.
- Traversing: To traverse a tree means to visit all the nodes in some specified order.
- Levels: The level of a particular node refers to how many generations the node is from the root. Root is assumed to be level 0.
- Keys: Key value is used to search for the item or perform other operations on it.

Tree Terminology

- A *tree* is a collection of elements (nodes)
- Each node may have 0 or more *successors*
  - (Unlike a list, which has 0 or 1 successor)
- Each node has *exactly one predecessor*
  - Except the starting / top node, called the *root*
- Links from node to its successors are called *branches*
- Successors of a node are called its *children*
- Predecessor of a node is called its *parent*
- Nodes with same parent are *siblings*
- Nodes with no children are called *leaves*

More terminology

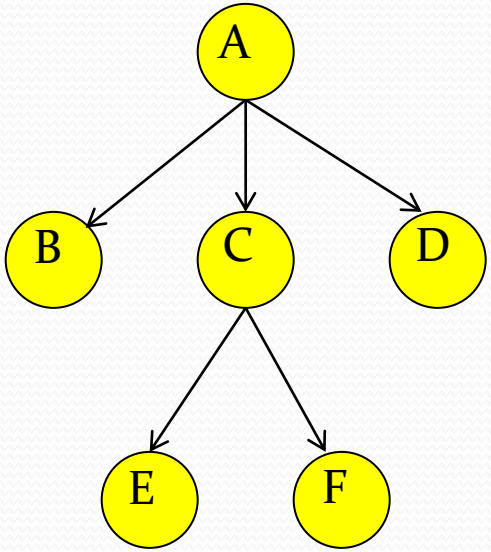
- Node A is the parent of node B if node B is a child of A
- Node A is an ancestor of node B if A is a parent of B, or if some child of A is an ancestor of B
  - In less formal terms, A is an ancestor of B if B is a child of A, or a child of a child of A, or a child of a child of a child of A, etc.
- Node B is a descendant of A if A is an ancestor of B
- Nodes A and B are siblings if they have the same parent

More Properties of Trees

- *depth:* the path length from the root of the tree to this node
- *height of a node:* The maximum distance (path length) of any leaf from this node
  - a leaf has a height of 0
  - the height of a tree is the height of the root of that tree
- *descendants*: any nodes that can be reached via 1 or more edges from this node
- *ancestors:* any nodes for which this node is a descendant

Tree

- Length of a path = number of edges
- Depth of a node N = length of path from root to N
- Height of node N = length of longest path from N to a leaf
- Level- level of root is 0



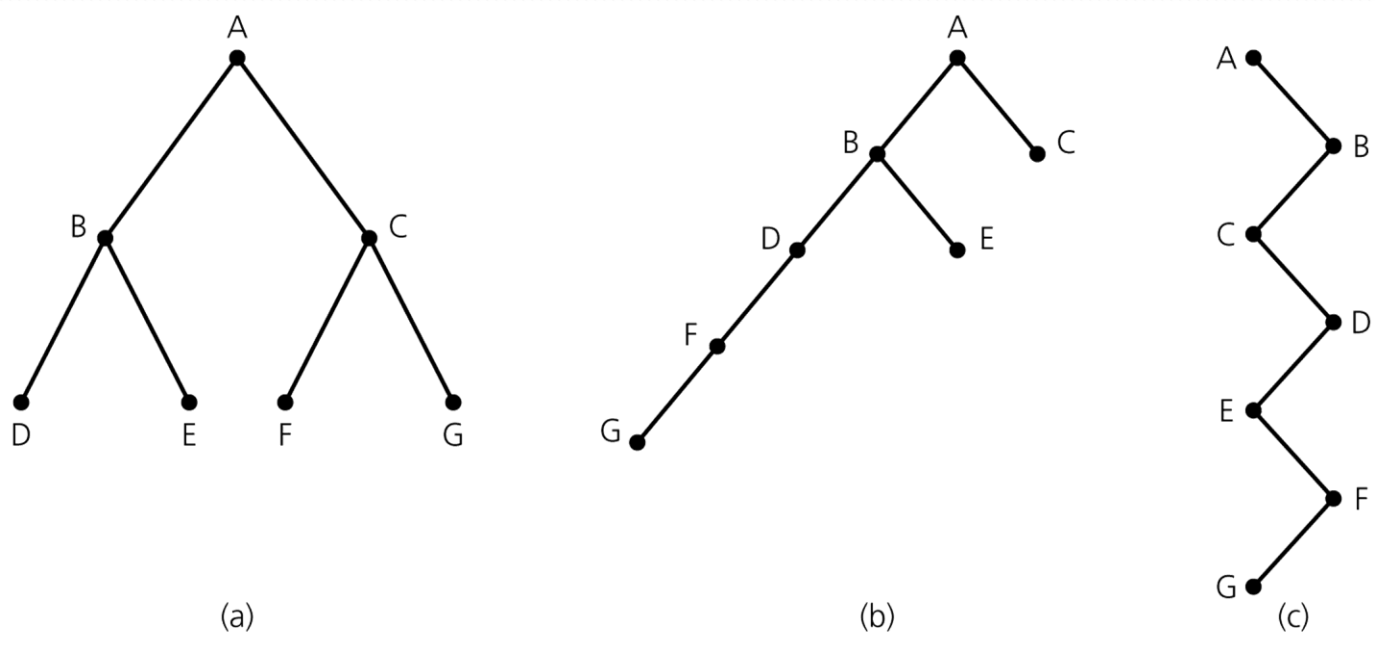
- ***The maximum number of nodes at level 'l' of a binary tree is 2^l.***

For root, l = 0, number of nodes = $2^0 = 1$

Assume that maximum number of nodes on level 'l' is 2^l

Height of Trees

- Trees come in many shapes



- Height of any tree: number of nodes on the longest path from the root to a leaf

Height

- height of a tree is maximum number of nodes on root to leaf path. Height of a tree with single node is considered as 1.
- ***Maximum number of nodes in a binary tree of height 'h' is $2^h - 1$.***
- **Maximum number of nodes in a binary tree of height h is $1 + 2 + 4 + .. + 2^{h-1}$**

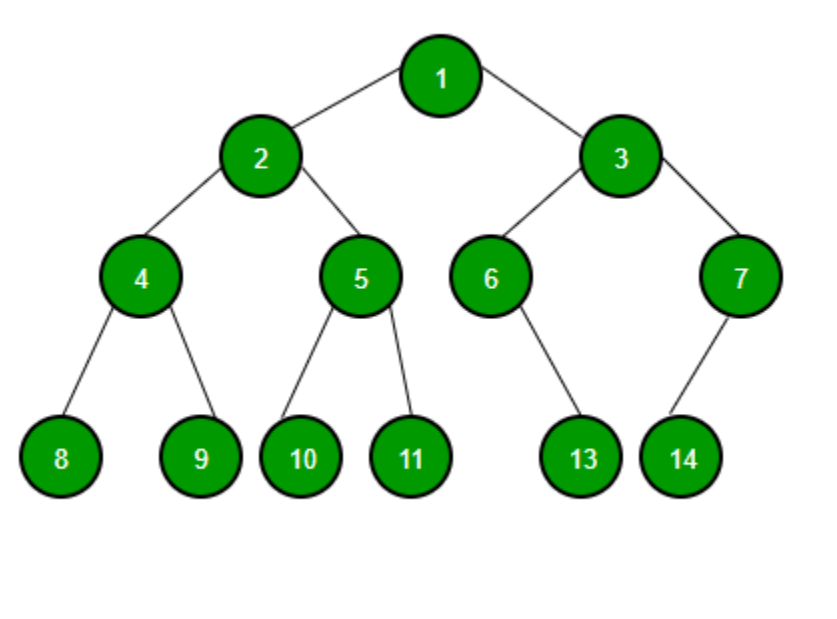
(height of the root is considered as 0. In this convention, the above formula becomes $2^{h+1} - 1$)

- ***In a Binary Tree with N nodes, minimum possible height or minimum number of levels is***

$Log_2(N+1)$

Binary Trees

- *Binary* tree: a node has *at most 2* non-empty subtrees
- Set of nodes T is a binary tree if either of these is true:
  - T is empty
  - Root of T has two subtrees, both binary trees
  - (Notice that this is a recursive definition)



Examples of Binary Trees

- *Expression tree*
  - Non-leaf (*internal*) nodes contain operators
  - Leaf nodes contain operands
- *Huffman tree*
  - Represents *Huffman codes* for characters appearing in a file or stream
  - Huffman code may use different numbers of bits to encode different characters
  - ASCII or Unicode uses a *fixed number* of bits for each character

Example of Binary Tree (2)

FIGURE 8.4
Expression Tree

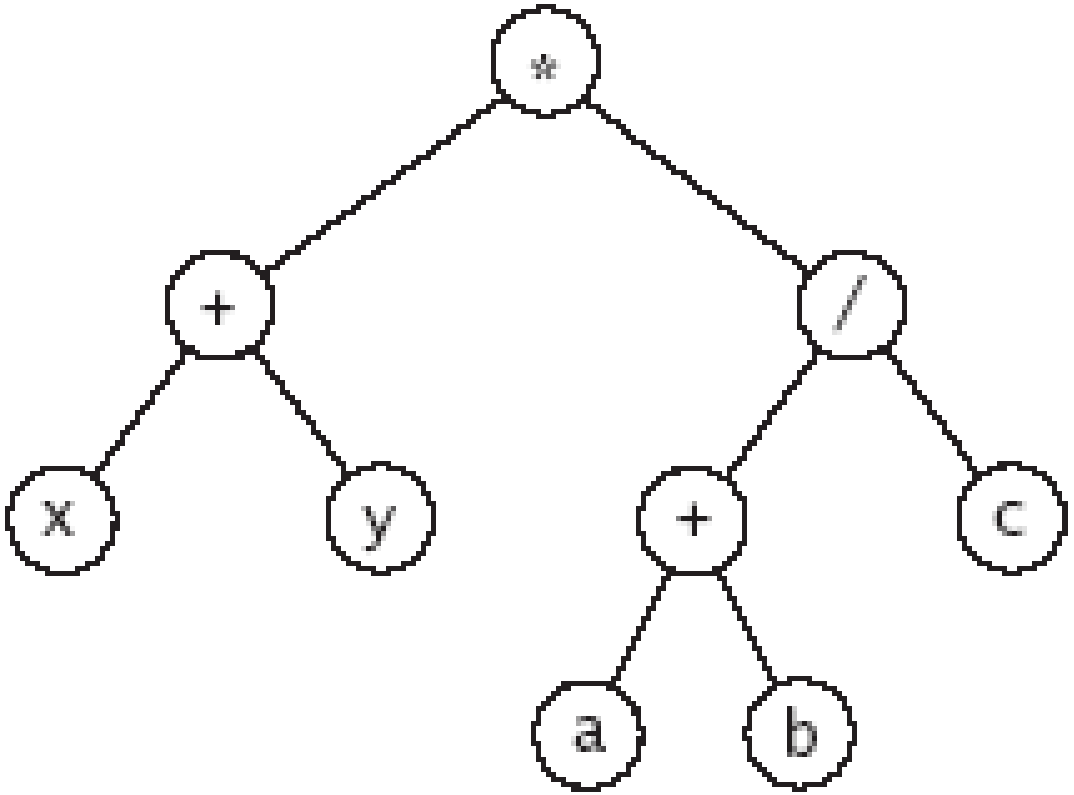
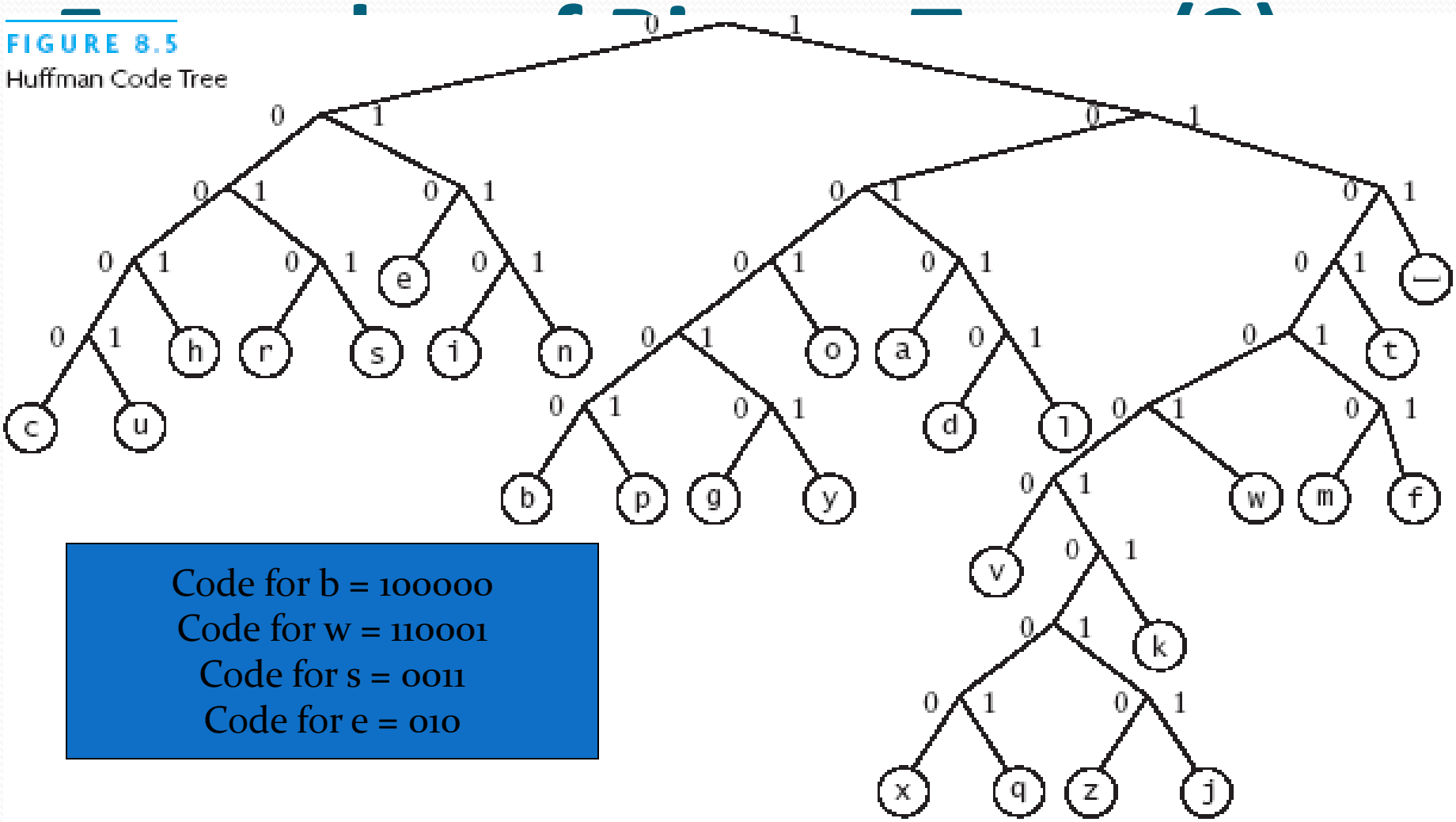


FIGURE 8.5

Huffman Code Tree

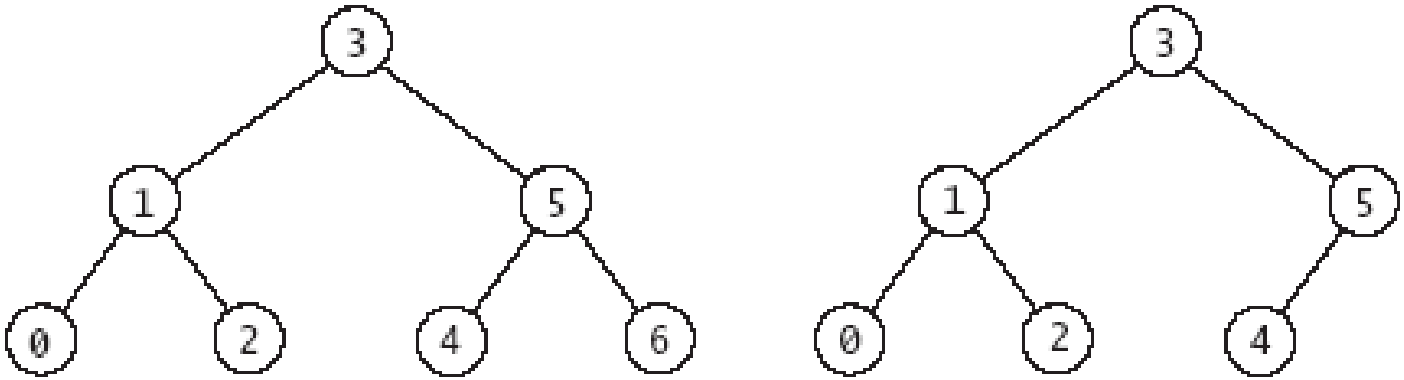


Code for b = 100000
Code for w = 110001
Code for s = 0011
Code for e = 010

Fullness and Completeness

- (In computer science) trees grow from the *top down*
- New values inserted in new leaf nodes
- A binary tree is *full* if all leaves are at the *same level*
- In a full tree, every node has 0 or 2 non-null children

FIGURE 8.6
Full Binary Tree (Left) and Complete Binary Tree (Right) of Height 3



- Full v.s. Complete Binary Trees. A full binary tree (sometimes proper binary tree or 2-tree) is a tree in which every node other than the leaves has two children. A complete binary tree is a binary tree in which every level, except possibly the last, is completely filled, and all nodes **are as far left as possible**.

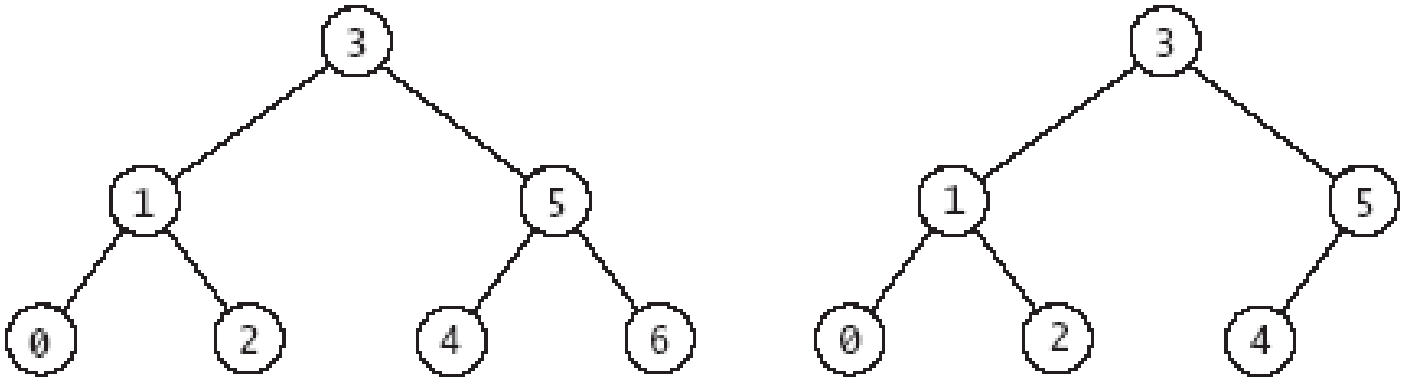
Full Binary Tree

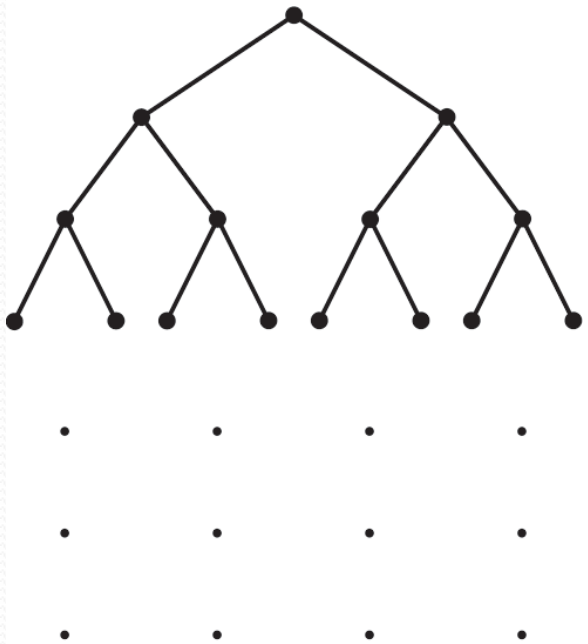
- If the height is $h>0$
  - The number of leaves is $2^{(h-1)}$
  - The number of nodes is $2^h - 1$
- If the number of nodes is $N>0$
  - The height is $\log_2(N+1)$
  - The number of leaves is $(N+1)/2$

Fullness and Completeness (2)

- A binary tree is <u>*complete*</u> if:
  - All leaves are at level *h* or level *h-1* (for some *h*)
  - All level *h-1* leaves are to the right

FIGURE 8.6
Full Binary Tree (Left) and Complete Binary Tree (Right) of Height 3





Level	Number of nodes at this level	Number of nodes at this and previous levels
1	$1 = 2^0$	$1 = 2^1 - 1$
2	$2 = 2^1$	$3 = 2^2 - 1$
3	$4 = 2^2$	$7 = 2^3 - 1$
4	$8 = 2^3$	$15 = 2^4 - 1$
•	•	•
•	•	•
•	•	•
h	2^{h-1}	$2^h - 1$

Balanced Binary Trees

- Balanced binary tree
  - The height of any node's right subtree differs from the height of the node's left subtree by no more than 1
  - A complete binary tree is balanced

Unbalanced Trees

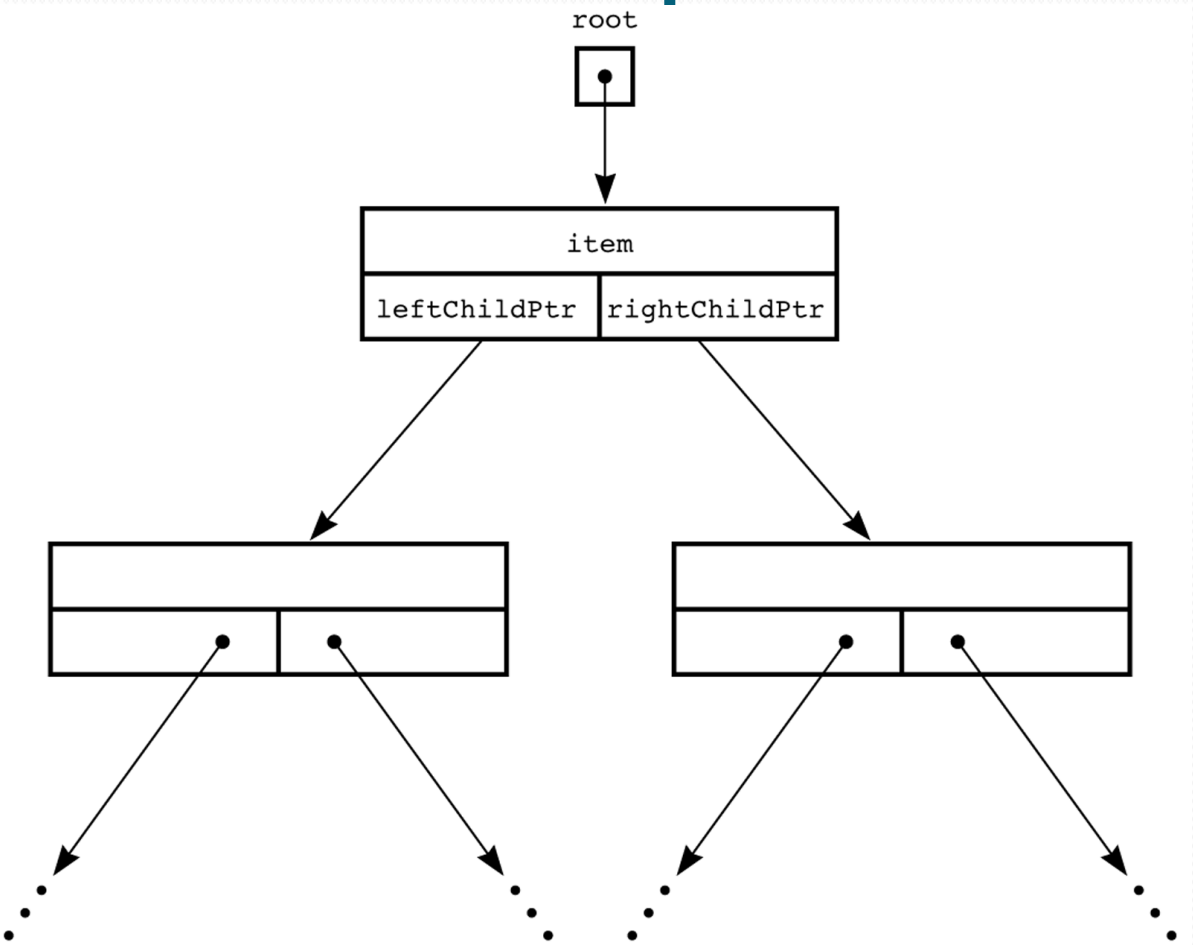
- Some trees can be unbalanced.
- They have most of their nodes on one side of the root or the other. Individual subtrees may also be unbalanced.
- Trees become unbalanced because of the order in which the data items are inserted.
- If the key values are inserted in ascending or descending order the tree will be unbalanced.
- For search-centric application (Binary tree), an unbalanced tree must be re-balanced.

Array-Based Implementation of a Complete Binary Tree

0	Jane
1	Bob
2	Tom
3	Alan
4	Ellen
5	Nancy
6	
7	

- If nodes numbered according to a level-by-level scheme
  - Root index: 0
  - Given any node tree[i]
    - Left child index: 2*i+1
    - Right chile index: 2*i+2
    - Parent index: (i-1)/2

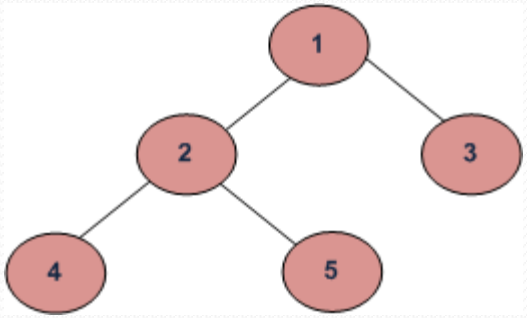
Pointer-Based Implementation



Traversals of Binary Trees

- Often want iterate over and process nodes of a tree
  - Can *walk* the tree and *visit* the nodes in order
  - This process is called ***tree traversal***
- Three kinds of binary tree traversal:
  - ***Pre***order(Root, Left, Right)
  - ***In***order(Left, Root, Right)
  - ***Post***order(Left, Right, Root)
- According to order of subtree ***root*** w.r.t. its ***children***

(a) Inorder (Left, Root, Right) : 4 2 5 1 3
(b) Preorder (Root, Left, Right) : 1 2 4 5 3
(c) Postorder (Left, Right, Root) : 4 5 2 3 1



InOrder(root) visits nodes in the following order:

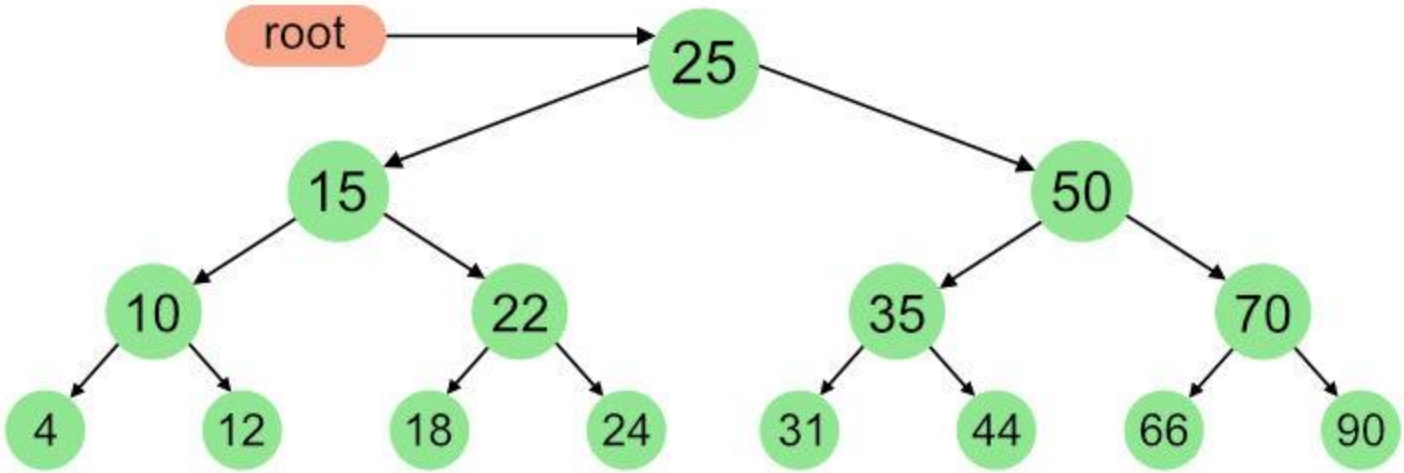
4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

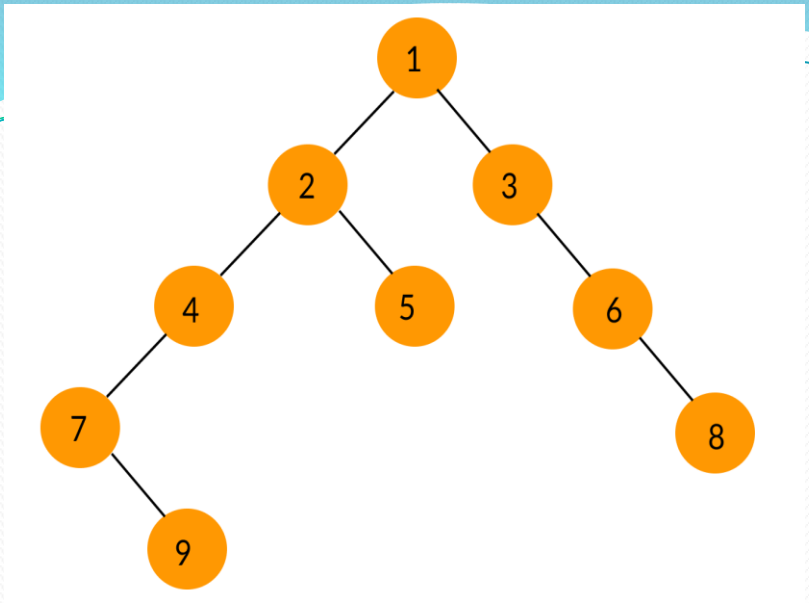
A Pre-order traversal visits nodes in the following order:

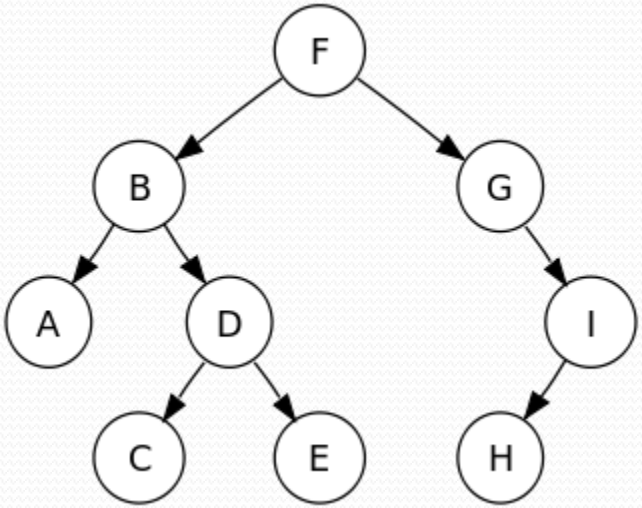
25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:

4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25




1
2
3
4
5
6
7
8
9


F
B
G
A
D
I
C
E
H

```
struct node
{
    int data;
    struct node* left;
    struct node* right;
};
```

```
struct node* newNode(int data)
{
    struct node* node = (struct node*)
                        malloc(sizeof(struct node));
    node->data = data;
    node->left = NULL;
    node->right = NULL;

    return(node);
}
```

```
void printInorder(struct node* node)
{
    if (node == NULL)
        return;

        printInorder(node->left);

        printf("%d ", node->data);

        printInorder(node->right);
}
```

```
void printPreorder(struct node* node)
{
    if (node == NULL)
        return;

        printf("%d ", node->data);

        printPreorder(node->left);


    printPreorder(node->right);
}
```

```
void printPostorder(struct node* node)
{
    if (node == NULL)
       return;

         printPostorder(node->left);


    printPostorder(node->right);


    printf("%d ", node->data);
}
```

Inorder Tree Traversal without Recursion

1) Create an empty stack S.

2) Initialize current node as root

3) Push the current node to S and set current = current->left until current is NULL

4) If current is NULL and stack is not empty then

a) Pop the top item from stack.

b) b) Print the popped item, set current = popped_item->right

c) c) Go to step 3.

5) If current is NULL and stack is empty then we are done.

```
          1
        /   \
      2       3
    /  \
  4     5
```

Step 1 Creates an empty stack: S = NULL

Step 2 sets current as address of root: current -> 1

Step 3 Pushes the current node and set current = current-

```
    current -> 1
    push 1: Stack S -> 1
    current -> 2
    push 2: Stack S -> 2, 1
    current -> 4
    push 4: Stack S -> 4, 2, 1
    current = NULL
```

Step 4 pops from S

```
    a) Pop 4: Stack S -> 2, 1
    b) print "4"
    c) current = NULL /*right of 4 */ and go to step 3
```

Since current is NULL step 3 doesn't do anything.

Step 4 pops again.

```
void inOrder(struct tNode *root)
{
   struct tNode *current = root;
  struct sNode *s = NULL;  /* Initialize stack s */
  bool done = 0;

  while (!done)
  {
      if(current !=  NULL)
    {
           push(&s, current);
     current = current->left;
    }

      else
    {
     if (!isEmpty(s))
     {
       current = pop(&s);
       printf("%d ", current->data);

              current = current->right;
     }
     else
       done = 1;
    }
  } /* end of while */
}
```

Postorder traversal without recursion

```
1.1 Create an empty stack
2.1 Do following while root is not NULL
    a) Push root's right child and then root to stack.
    b) Set root as root's left child.
2.2 Pop an item from stack and set it as root.
    a) If the popped item has a right child and the right child
       is at top of stack, then remove the right child from stack,
       push the root back and set root as root's right child.
    b) Else print root's data and set root as NULL.
2.3 Repeat steps 2.1 and 2.2 while stack is not empty.
```